

End-to-end NLP System Building

Gefei Gu

gefeig@andrew.cmu.edu

Yi Lu

yilu4@andrew.cmu.edu

Yi Dai

yidai@andrew.cmu.edu

Abstract

We build an end-to-end retrieval-augmented generation system for factual question answering about Pittsburgh and Carnegie Mellon University. Our pipeline combines a custom document chunker that splits crawled web data into coherent segments, and a hybrid retriever that integrates dense (FAISS + all-MiniLM-L6-v2) and sparse (BM25) methods using reciprocal rank fusion. The retrieved passages are passed to a Qwen2.5-Instruct model for answer generation. Experiments show that the hybrid retrieval strategy and hybrid chunking significantly improve factual accuracy compared to closed-book baselines.

1 Introduction

In assignment 2, our group builds an RAG system capable of answering factual questions about Pittsburgh and CMU. The entire pipeline was implemented from scratch, including data collection from the website, document preprocessing, chunking(chunk.py), retrieval, and answer generation(rag.py). We first collected and cleaned a large set of Pittsburgh- and CMU-related web documents and PDFs, covering various domains, like history, events, food, music, and sports. These resources were then processed into manageable chunks using a flexible document chunker that supports multiple strategies (naive, sentence, paragraph, and hybrid).

For retrieval, both dense and sparse methods were implemented. Dense retrieval was performed using all-MiniLM-L6-v2 embeddings with FAISS for semantic search, while sparse retrieval employed BM25 for keyword-based ranking. Additionally, a hybrid retrieval approach based on Reciprocal Rank Fusion (RRF) was used to combine the strengths of both methods. The top-ranked passages are then provided as contextual input to the generation models (Qwen-2.5-7B-Instruct and Qwen-2.5-32B-Instruct) to produce factual answers.

In general, our project constructs an end-to-end RAG system that can answer questions about Pittsburgh and CMU. Moreover, we explore how different document chunking and retrieval strategies can affect factual QA performance in domain-specific RAG systems. The following sections describe our data creating process, implementation details, and experiment results.

2 Data Creation

2.1 Knowledge Resource Compilation

We collected publicly available documents relevant to Pittsburgh and CMU to compile the knowledge resource. The collected documents were classified into five categories: calendar, food, music & art, sport, and general information. For the calendar category, we focused on capturing concise information about events, including annual or recurring events, as well as those occurring from October 9th, 2025, to December 31st, 2025. For the food, music & art, sport, and general information categories, we primarily considered subpages such as “about,” “schedule,” and “history.” For other subpages, the scope of crawling was controlled by two constraints: (1) the recursive depth was limited to 2, and (2) the total number of pages per source was capped at 50 to ensure relevance.

2.2 Extraction Method

The knowledge resource consists of webpages and PDF documents. For static webpages, Python libraries such as requests and BeautifulSoup¹ were used to crawl and extract content. For pages requiring dynamic content loading, Selenium² was employed to simulate user interactions and capture the fully rendered HTML. Additionally, some pages loaded content via AJAX, which was handled by sending direct GET requests and parsing

¹<https://pypi.org/project/beautifulsoup4/>

²<https://www.selenium.dev/>

the responses. The extracted webpage content was cleaned to remove HTML tags, leaving only plain text. For PDF documents, we used pdfplumber³ to extract textual content. Finally, all information was processed and saved as Markdown and plain text files. In total, X documents were collected.

2.3 Data Annotation

We first manually constructed 287 question–answer samples. Each sample contained a category, a question, an answer, and its supporting context, which were used to verify and refine the answers. Three annotators participated in the annotation process. One annotator was responsible for carefully selecting 150 high-quality samples (30 per category, covering calendar, food, music & art, sport, and general information) from the 287 samples to ensure data diversity. During this process, we used Label Studio⁴ as the annotation interface to efficiently validate annotations and filter high-quality data. Each sample was finally assigned one of two labels: *accept* or *reject*. Table 1 shows the sample questions and their corresponding reference answers.

To evaluate annotation quality, the other two annotators also annotated 100 randomly selected samples from the 287-sample set. We computed Krippendorff’s alpha (Krippendorff, 2004), defined as:

$$\alpha = 1 - \frac{D_o}{D_e} \quad (1)$$

where D_o is the observed disagreement among annotations, and D_e is the expected disagreement if the annotations were random. Our calculated alpha was 0.8241, indicating a high level of agreement among annotators.

For the final set of 150 high-quality samples, we randomly selected 45 for additional annotation. Given that Krippendorff’s alpha may be unreliable for highly imbalanced binary data, we directly compared the annotations of the three annotation results, and they were completely consistent.

3 Implementation

3.1 Document Chunking

Due to the maximum context length constraints of LLMs and to enhance the accuracy of retrieval, we divided the documents into smaller chunks using a custom-built document chunker. The chunker

is designed to support multiple splitting strategies, including: fixed-length splitting, custom separator splitting, markdown header splitting.

Naive Fixed-Length Splitting. This strategy splits the text into chunks of a maximum size `chunk_size`. It also uses the `chunk_overlap` parameter to help maintain context continuity across chunks.

Custom Separator Splitting. This strategy is an extension of the naive approach, allowing the user to specify custom separators to split the text. For example, paragraph-based splitting divides the text using double newline characters (`\n\n`), while sentence-based splitting divides the text using punctuation marks. This strategy also supports the `chunk_overlap` parameter.

Markdown Header Splitting. Markdown documents are inherently structured with headers, making it easy to split the text. Therefore, we first divide the document based on top-level and second-level headers (`#`, `##`). For sections that are longer than the maximum chunk size, we further apply the naive fixed-length splitting.

After applying the chunking strategy, we generate a JSON file for each document, which contains a list of chunks.

3.2 Retriever

3.2.1 Dense Retrieval

We use all-MiniLM-L6-v2 as the embedding model to encode each document chunk into a 384-dimensional dense vector. All vectors are then L2-normalized to ensure stable computations. Next, we construct a FAISS (Douze et al., 2024) index using IndexFlatIP, where inner product is used to measure the similarity between vectors:

$$\text{Inner Product}(\mathbf{u}, \mathbf{v}) = \sum_i u_i v_i \quad (2)$$

Finally, all document vectors are added to the FAISS index. During retrieval, a new query is encoded and normalized in the same way, and the FAISS index is used to search for the top- k most similar document chunks.

3.2.2 Sparse Retrieval

We use the BM25Okapi implementation from rank-bm25⁵. Each document chunk is first tokenized by lowercasing and splitting on whitespace. BM25 then indexes the tokenized chunks and computes a

³<https://github.com/jsvine/pdfplumber>

⁴<https://labelstud.io/>

⁵<https://pypi.org/project/rank-bm25/>

Category	Question	Reference Answer
calendar	When is Carnegie Mellon University’s Homecoming Weekend celebrating CMU’s 125th anniversary?	November 6-8
food	In which year was Pittsburgh’s Soul Food Festival established?	2019
general information	Where does Carnegie Mellon University have campuses?	Carnegie Mellon University has campuses in Pittsburgh, Silicon Valley, Qatar, and Africa.
music&art	What types of artifact donations does the Heinz History Center seek?	Artifacts and archival materials related to the history of Western Pennsylvania and the experiences of its people, both past and present.
sport	Who won the Pittsburgh Steelers team MVP award in 2024?	T.J. Watt

Table 1: Sample Questions and Reference Answers

relevance score for each term based on its term frequency, inverse document frequency, and document length normalization. The user query is similarly tokenized and scored against the document index. BM25 returns a relevance score for each document chunk, and the top- k chunks with the highest scores are selected as retrieval results.

3.2.3 Combination Strategy

Since BM25 is effective in capturing exact keyword matches, whereas dense retrieval excels at modeling semantic relevance, we adopt Reciprocal Rank Fusion (RRF)(Cormack et al., 2009) to combine their ranking results. The key idea of RRF is to assign each document a combined score based on its rank positions across different retrieval systems. The scoring function is formally defined as:

$$\text{score}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (3)$$

where $\text{rank}_r(d)$ denotes the rank position of document d in retrieval system r , and k is a smoothing constant.

3.3 Generator

We use two different LLMs for generation (Qwen-2.5-7B-Instruct and Qwen-2.5-32B-Instruct)(Qwen et al., 2025) both of which satisfy the requirement that the model must have been released before January 1, 2025, and contain no more than 32 billion parameters. We use vLLM(Kwon et al., 2023) to serve these models for inference. We provide our prompts in Appendix A.

4 Experiment Setting

We explore the performance of different chunking strategies under various retrieval strategies.

Document Chunking. The maximum chunk size is 1000 tokens, and the overlap is 200 tokens.

Retrieval and Ranking. The top-5 document chunks are retained. These chunks are subsequently re-ranked using RRF with a hyperparameter $k = 60$.

Model Details. Embeddings are generated using the all-MiniLM-L6-v2 model. Final answers are produced by Qwen-2.5-32B-Instruct and Qwen-2.5-7B-Instruct.

Evaluation Metrics. Performance is measured using Recall, Exact Match (EM), and F1 score. The calculation follows the method described in the SQuAD(Rajpurkar et al., 2016).

5 Experiment Results

5.1 retrieve-and-augment strategy vs closed-book use of your models

We evaluated Qwen-2.5-7B-Instruct on our test dataset in a closed-book setting, where it achieved an F1 score of 0.1987—significantly lower than the score obtained with RAG (0.5383). This result suggests that the questions in our dataset are challenging for the model to answer using its internal knowledge alone, and that retrieval augmentation provides substantial improvement.

Chunk Strategy	Retriever Strategy	EM	F1	Recall
naive	hybrid	0.3377	<u>0.5236</u>	<u>0.3974</u>
	dense	0.2848	0.4768	0.3444
	sparse	0.2980	0.4653	0.3444
paragraph	hybrid	0.3311	0.5230	0.4103
	dense	0.2781	0.4647	0.3709
	sparse	0.3377	0.5072	0.3841
sentence	hybrid	0.2715	0.4158	0.3245
	dense	0.2119	0.3560	0.2649
	sparse	0.2053	0.3281	0.2384
hybrid	hybrid	<u>0.3367</u>	0.5383	0.4106
	dense	0.2583	0.4438	0.3444
	sparse	0.3245	0.4994	0.3775

Table 2: Performance comparison across chunking and retriever strategies. Bold indicates the best value in each column; underlined indicates the second-best value (ties included).

Table 3: F1 performance comparison across different models and retrieval strategies (Top-K=5)

Model	Strategy	F1 Score
Qwen2.5-7B	hybrid	0.5383
	sparse	0.4994
	dense	0.4438
Qwen2.5-32B	hybrid	0.5642
	sparse	0.5153
	dense	0.4898

tive than using paragraph-based or naive text splitters. Comparing the results of paragraph-based and naive strategies, the naive strategy performs slightly worse, suggesting that fixed-length chunking may compromise semantic integrity, while paragraph-based chunking can preserve semantic completeness to some extent. Sentence-level chunking performs the worst across EM, F1, and Recall metrics, further confirming that overly fine-grained chunking disrupts context and negatively affects retrieval performance.

5.2 Chunking Strategy Analysis

We compared four chunking strategies: naive, paragraph, sentence, and hybrid. The naive strategy splits both TXT and Markdown documents into fixed-length chunks. The paragraph strategy divides TXT and Markdown documents using two consecutive newline characters (`\n\n`) as separators. The sentence strategy segments TXT and Markdown documents based on sentence-ending punctuation. The hybrid strategy uses our custom Markdown header splitter to split Markdown documents, while applying paragraph-based splitting only to TXT documents. All strategies used a unified setting of `chunk_size=1000` and `overlap = 200`.

The results are shown in the Table 2. Overall, hybrid chunking achieves the highest F1-score and recall (0.5383 and 0.4106, respectively), indicating that for Markdown documents, employing a Markdown header-based splitter is more effective

5.3 Retrieval Strategy Analysis

In this experiment, we compared three retrieval strategies: dense-only, sparse-only, and hybrid, and set top- k to 5 for all methods. The dense-only method directly retrieves the top- k semantically most similar chunks from the FAISS index, while the sparse-only method uses BM25 to return the top- k highest-scoring chunks from all available chunks. For hybrid retrieval, both the dense and sparse components first return top- $k \times 2$ candidate chunks. These candidates are then fused using RRF with the hyperparameter k set to 60, and the top- k chunks are selected as the final retrieval contexts.

The results from Table 2 indicate that hybrid retrieval achieves the best performance across all chunking strategies. By combining semantic matching with keyword coverage, hybrid retrieval is able to maintain high retrieval effectiveness across text chunks of varying granularity.

5.4 Model Size Analysis

We use two models from the same family but with different model sizes to investigate how model capacity affects the accuracy of final answer generation in our RAG pipeline. Results are shown in Table 3. The larger model (Qwen2.5-32B) consistently outperforms the smaller model (Qwen2.5-7B) across all three retrieval strategies. Specifically, the 32B model achieves higher F1 scores (0.5642 vs. 0.5383), demonstrating that increased model capacity leads to better answer generation quality.

6 Conclusion

Through experimenting with different chunking methods and retrieval strategies, we found that a hybrid chunking strategy combined with a hybrid retrieval approach achieves the best performance. We also investigated how model capacity affects the accuracy of final answer generation in our RAG pipeline and found that larger models lead to higher-quality answers. For future work, we plan to expand the types of QA tasks beyond factual questions, including multi-hop and other complex question-answering scenarios. In addition, the high-quality outputs generated by the 32B model could be leveraged to fine-tune smaller models, enabling them to produce more informative responses. Finally, evaluation metrics for RAG could be diversified beyond EM, F1, and Recall; for example, metrics specifically designed for RAG, such as RAGChecker(Ru et al., 2024), could provide a more comprehensive assessment of model performance.

References

- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *SIGIR*, pages 758–759. ACM.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. [The faiss library](#).
- Klaus Krippendorff. 2004. [Reliability in content analysis: Some common misconceptions and recommendations](#). *Human Communication Research*, 30(3):411–433.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. [Efficient memory management for large language](#)

[model serving with pagedattention](#). *Preprint*, arXiv:2309.06180.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100, 000+ questions for machine comprehension of text. In *EMNLP*, pages 2383–2392. The Association for Computational Linguistics.

Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, Cheng Jiayang, Cunxiang Wang, Shichao Sun, Huanyu Li, Zizhao Zhang, Binjie Wang, Jiarong Jiang, Tong He, Zhiguo Wang, Pengfei Liu, Yue Zhang, and Zheng Zhang. 2024. Ragchecker: A fine-grained framework for diagnosing retrieval-augmented generation. In *NeurIPS*.

A Prompt for generation

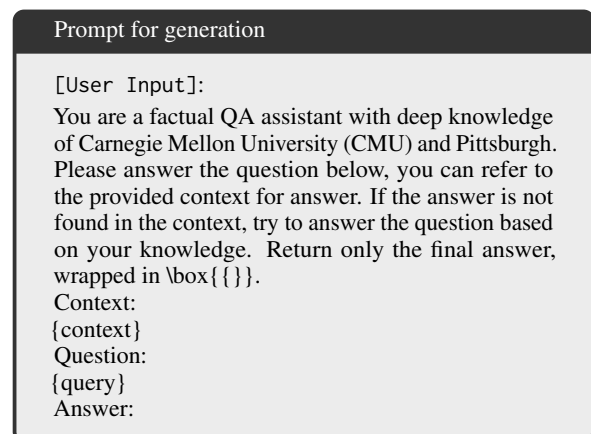


Figure 1: Example of prompt for generation.